МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу «Дискретный анализ»
на тему « Сбалансированные деревья»

Выполнил: Н.В. Храмов
Группа: М8О-208БВ-24
Преподаватель: Макаров Н.К.
Проверил: Сахарин Н.А.
Дата: 14.04.2026
Оценка:
Подпись:

Москва, 2025

Постановка задачи

Необходимо создать программную библиотеку, реализующую указанную структуру данных, на основе которой разработать программу-словарь. В словаре каждому ключу, представляющему из себя регистронезависимую последовательность букв английского алфавита длиной не более 256 символов, поставлен в соответствие некоторый номер, от 0 до $2^{64}-1$. Разным словам может быть поставлен в соответствие один и тот же номер.

Программа должна обрабатывать строки входного файла до его окончания. Каждая строка может иметь следующий формат:

+ word 34 — добавить слово «word» с номером 34 в словарь. Программа должна вывести строку «OK», если операция прошла успешно, «Exist», если слово уже находится в словаре.

- word — удалить слово «word» из словаря. Программа должна вывести «OK», если слово существовало и было удалено, «NoSuchWord», если слово в словаре не было найдено.

word — найти в словаре слово «word». Программа должна вывести «OK: 34», если слово было найдено; число, которое следует за «OK:» — номер, присвоенный слову при добавлении. В случае, если слово в словаре не было обнаружено, нужно вывести строку «NoSuchWord».

Для всех операций, в случае возникновения системной ошибки (нехватка памяти, отсутсвие прав записи и т.п.), программа должна вывести строку, начинающуюся с «ERROR:» и описывающую на английском языке возникшую ошибку.

Структура данных: Декартово дерево.

Формат ввода

На каждой непустой строке входного файла задаётся команда:

- + word number — добавить пару (слово, число)
- - word — удалить слово
- word — найти значение по слову

Слова обрабатываются без учёта регистра.

Формат вывода

Для каждой команды выводится результат:

- OK — операция выполнена успешно
- Exist — слово уже есть в словаре
- NoSuchWord — слово отсутствует
- OK: value — результат поиска

Алгоритм решения задачи

Организация входных данных. Программа читает данные из стандартного потока ввода (std::cin) построчно с помощью std::getline до достижения конца файла (EOF). Каждая строка входного файла представляет собой одну команду и обрабатывается независимо. Форматы команд строго соответствуют требованиям задания:

- + word число — вставка слова с указанным номером;
- - word — удаление слова;
- word — поиск слова.

Слово word приводится к нижнему регистру (нормализация) и рассматривается как ключ длиной не более 256 символов английского алфавита. Номер — целое число от 0 до $2^{64}-1$.

Структура данных.

В качестве основной структуры данных реализовано **декартово дерево (Treap)**. Каждый узел содержит:

- key — нормализованное слово (std::string);
- val — соответствующий номер (uint64_t);
- prio — случайный приоритет (uint64_t, генерируется с помощью mt19937_64);
- указатели left, right и parent.

Использование указателя parent позволяет выполнять все операции итеративно. Сравнение ключей производится через объект Compare = std::less<std::string>. Дерево поддерживает свойства BST по ключам и heap по приоритетам.

Вставка элемента. Вставка выполняется итеративно в два этапа. Сначала осуществляется поиск позиции вставки как в обычном бинарном дереве поиска. Новый узел добавляется в качестве листа. Затем выполняется подъём узла вверх по дереву: пока приоритет нового узла больше приоритета его родителя, производится соответствующий поворот (rotate_right или rotate_left). Все повороты выполняются итеративно с обновлением указателей parent. Операция использует только константный дополнительный объём памяти.

Удаление элемента. Удаление также полностью итеративно. Сначала производится поиск узла по ключу. Найденный узел опускается вниз до положения листа: на каждом шаге сравниваются приоритеты левого и правого потомков, и выполняется поворот в сторону потомка с большим приоритетом. После того как узел становится листом (не имеет детей), он отсоединяется от родителя и освобождается. При отсутствии элемента выводится сообщение «NoSuchWord».

Поиск элемента. Поиск осуществляется итеративным спуском по дереву от корня, используя сравнение ключей через Compare. При достижении узла с совпадающим ключом возвращается его значение val. Если узел не найден — возвращается признак отсутствия элемента. Операция не использует рекурсию и дополнительную память, зависящую от высоты дерева.

Анализ сложности

Временная сложность операций.

Все операции реализованы итеративно с использованием указателей на родителей и поворотов (rotate_left / rotate_right). Благодаря случайным приоритетам декартово дерево (Treap) обладает следующими характеристиками:

- **Поиск элемента** — O(log n) в среднем (ожидаемая сложность), O(n) в худшем случае.
- **Вставка элемента** — O(log n) в среднем (ожидаемая сложность), O(n) в худшем случае.
- **Удаление элемента** — O(log n) в среднем (ожидаемая сложность), O(n) в худшем случае.

Каждая операция использует только константный дополнительный объём памяти (O(1)) и не содержит рекурсии.

Итоговая временная сложность. Программа обрабатывает входной файл построчно до достижения конца файла. При общем количестве операций M и максимальном размере словаря N итоговая временная сложность программы составляет **O(M log N)** в среднем случае (по ожидаемой сложности операций над Treap). В худшем случае (теоретически) — O(M·N), однако вероятность такого случая экспоненциально мала благодаря случайной генерации приоритетов.

Исходный код

```
#include <bits/stdc++.h>

using namespace std;

struct Node {
    string key;
    uint64_t val;
    uint64_t prio;
    Node* left = nullptr;
    Node* right = nullptr;
    Node* parent = nullptr;

    Node(const string& k, uint64_t v) : key(k), val(v),
left(nullptr), right(nullptr), parent(nullptr) {
        static mt19937_64 generator(random_device{}());
        prio = generator();
    }
};

class Treap {
private:
    Node* root = nullptr;

    using Compare = less<string>;
    Compare comp;

    void rotate_right(Node* x) {
        if (!x || !x->left) return;
        Node* y = x->left;
        Node* p = x->parent;

        x->left = y->right;
        if (y->right) y->right->parent = x;

        y->right = x;
        x->parent = y;
        y->parent = p;
```

```
        if (p) {
            if (p->left == x) p->left = y;
            else p->right = y;
        } else {
            root = y;
        }
    }

    void rotate_left(Node* x) {
        if (!x || !x->right) return;
        Node* y = x->right;
        Node* p = x->parent;

        x->right = y->left;
        if (y->left) y->left->parent = x;

        y->left = x;
        x->parent = y;
        y->parent = p;

        if (p) {
            if (p->left == x) p->left = y;
            else p->right = y;
        } else {
            root = y;
        }
    }

    Node* search(const string& key) const {
        Node* cur = root;
        while (cur) {
            if (!comp(key, cur->key) && !comp(cur->key, key)) {
                return cur;
            }
            if (comp(key, cur->key)) {
                cur = cur->left;
            } else {
                cur = cur->right;
            }
        }
        return nullptr;
    }

public:
    Treap() = default;

    bool insert(const string& key, uint64_t v) {
        if (search(key)) return false;

        if (!root) {
            root = new Node(key, v);
            return true;
        }
```

```
        Node* cur = root;
        Node* par = nullptr;
        bool go_left = false;
        while (cur) {
            par = cur;
            if (comp(key, cur->key)) {
                go_left = true;
                cur = cur->left;
            } else {
                go_left = false;
                cur = cur->right;
            }
        }

        Node* newn = new Node(key, v);
        newn->parent = par;
        if (go_left) par->left = newn;
        else par->right = newn;

        Node* node = newn;
        while (node->parent) {
            Node* p = node->parent;
            if (node->prio <= p->prio) break;
            if (p->left == node) {
                rotate_right(p);
            } else {
                rotate_left(p);
            }
        }
        return true;
    }

    bool erase(const string& key) {
        Node* node = search(key);
        if (!node) return false;

        while (node->left || node->right) {
            uint64_t left_p = node->left ? node->left->prio :
0ULL;
            uint64_t right_p = node->right ? node->right->prio :
0ULL;
            if (left_p > right_p) {
                rotate_right(node);
            } else if (right_p > 0ULL) {
                rotate_left(node);
            } else {
                break;
            }
        }

        Node* p = node->parent;
        if (p) {
            if (p->left == node) p->left = nullptr;
            else p->right = nullptr;
```

```
        } else {
            root = nullptr;
        }
        delete node;
        return true;
    }

    pair<bool, uint64_t> get(const string& key) const {
        Node* n = search(key);
        if (n) return {true, n->val};
        return {false, 0ULL};
    }
};

string normalize(string s) {
    for (char& c : s) {
        c = tolower(static_cast<unsigned char>(c));
    }
    return s;
}

int main() {
    Treap dict;
    string line;
    while (getline(cin, line)) {
        if (line.empty()) continue;
        istringstream ss(line);
        string token;
        if (!(ss >> token)) continue;

        if (token == "+") {
            string w;
            uint64_t num;
            if (ss >> w >> num) {
                string key = normalize(w);
                if (dict.insert(key, num)) {
                    cout << "OK" << endl;
                } else {
                    cout << "Exist" << endl;
                }
            }
        } else if (token == "-") {
            string w;
            if (ss >> w) {
                string key = normalize(w);
                if (dict.erase(key)) {
                    cout << "OK" << endl;
                } else {
                    cout << "NoSuchWord" << endl;
                }
            }
        } else {
            string w = token;
            string key = normalize(w);
```

```
            auto [found, num] = dict.get(key);
            if (found) {
                cout << "OK: " << num << endl;
            } else {
                cout << "NoSuchWord" << endl;
            }
        }
    }
    return 0;
}
```

Тесты

Тест 1
>>>
+ apple 10
apple
<<<
OK
OK: 10

Тест 2
>>>
+ apple 10
+ apple 20
<<<
OK
Exist

Тест 3
>>>
+ apple 10
- apple
apple
<<<
OK
OK
NoSuchWord

Тест 4
>>>
+ apple 10
+ banana 20
banana
<<<
OK
OK
OK: 20

Тест 5
>>>
+ Apple 10
apple
<<<
OK
OK: 10

Вывод

В рамках выполнения задания полностью реализована программная библиотека декартового дерева (Treap) с итеративными операциями поиска, вставки и удаления элементов. На основе данной библиотеки разработана программа-словарь, обрабатывающая команды в требуемом формате и обеспечивающая регистронезависимую работу с ключами.

Все основные требования задания выполнены:

- отсутствует рекурсия во всех методах;
- операции поиска, вставки и удаления используют только константный дополнительный объём памяти (O(1));
- реализация Treap выполнена исключительно через повороты (rotate), без использования split/merge;
- сравнение ключей осуществляется через Compare = std::less<std::string>.

Корректность работы программы подтверждена набором тестов, покрывающих все возможные сценарии операций. Средняя временная сложность каждой операции составляет **O(log N)**, что обеспечивает эффективную обработку входных данных.